

Final Project Approach

M5 Retail Demand Forecasting — Problem Statement 11

Senior Review, Discrepancy Resolution & Team-Ready ML Strategy

NPN AIA Hackathon — St. Joseph's College of Engineering

Senior Review, Discrepancy Resolution & Team-Ready ML Strategy

NPN AIA Hackathon — St. Joseph's College of Engineering

Review stage. No model has been trained, no feature dataset has been built, and no file in raw_dataset/, processed_dataset/, EDA/, Or Project_Approach/ was modified while preparing this document.

How to read this document

This document reviews everything the team has produced so far (EDA/, Project_Approach/, processed_dataset/, raw_dataset/, analysis_output/), resolves every conflicting number that has come up in team discussion, and gives one final, opinionated, hackathon-feasible ML strategy. Where a claim is **FACT**, it was directly verified — either by an existing report, or independently recomputed for this document straight from raw_dataset/ and processed_dataset/sales_long_full.parquet (see SUPPORTING_EVIDENCE.md for the exact verification commands and outputs). Where something is **INFERENCE**, it is a reasonable but not directly provable interpretation. Where something is an **ASSUMPTION**, it is a choice we are making for modeling purposes, not a fact about the world.

Every technical term is explained in plain English the first time it appears.

Executive Summary

We are forecasting daily unit sales for **30,490 store-item series** (3,049 products × 10 stores) for **28 days** (2016-05-23 → 2016-06-19), using 1,941 days (~5.3 years) of real history. The dataset is the public M5 Forecasting — Accuracy dataset (Walmart), and the team's own processing/EDA work on it is sound: the processed table (sales_long_full.parquet, 59,181,090 rows × 22 columns) was independently re-verified for this review directly against raw_dataset/ and matches exactly.

The dominant fact about this data is zero-inflation: 68.0% of all rows are zero sales. A second, load-bearing fact is that a large share of those zeros are not ordinary "no one bought it today" — they are **pre-listing zeros**, caused by a product not yet being stocked in a given store. The EDA found strong, specific evidence for this (leading zero-block length lines up with leading no-price-history length for 99.48% of series, median gap 3 days). Neither fact is new — both are already well documented in EDA/EDA_REPORT.md — but they are the two facts that should drive every subsequent modeling decision, and this document holds the team to that.

Four previously-discussed numbers turned out to be either resolvable errors or apples-to-oranges comparisons, not genuine unresolved conflicts. All four are traced to their root cause in Step 1 / SUPPORTING_EVIDENCE.md. Short version:

Disputed pair	Verdict
634 vs. 763 max daily sales	763 is correct (independently re-verified 3 ways). 634 is a *scoped* maximum — the max within department FOODS_2, and coincidentally also the max within store TX_1 — not the dataset-wide max.

Disputed pair	Verdict
~30M vs. 59,181,090 processed rows	59,181,090 is correct and exact (30,490 series × 1,941 days, independently re-verified). "~30M" appears nowhere in any project file — it looks like a units slip confusing the *series count* (30,490) with a *row count*.
69.56% vs. 68.6% FOODS sales share	68.6% (precisely 68.62%) is correct — independently reproduced three separate ways (raw evaluation file, raw validation file, processed parquet). No definition tested (revenue-weighted, validation-file-only, nonzero-row-weighted) reproduces 69.56%; it does not appear to be traceable to this dataset and should be dropped.
42,840 vs. 30,490 series	Both numbers are individually correct, but they answer different questions. 30,490 is the actual forecasting granularity (store × item) — verified directly, and it's what <code>sample_submission.csv</code> requires. 42,840 is the well-known public M5 fact that the WRMSSE metric's hierarchy sums to 42,840 series across all 12 aggregation levels (1 total + 3 state + 10 store + 3 category + 7 dept + 9 state×cat + 21 state×dept + 30 store×cat + 70 store×dept + 3,049 item + 9,147 state×item + 30,490 store×item). It is not the number of things this project must forecast.

Recommended strategy, in one paragraph: Build a global LightGBM baseline first (single model, standard features, Tweedie loss) and get it through a real fixed-origin backtest before anything else. Only after that baseline exists and is measured, build the two-stage "hurdle" model ($P(\text{sale}) \times E(\text{units} \mid \text{sale})$) with recency-state features (`days_since_last_sale`, `zero_streak_length`) and a listing-aware pre-listing flag, and prove via ablation that each addition earns its keep. The team's proposed novelty — **"Listing-Aware + Recency-Aware Demand Forecasting"** — is defensible and should remain the headline story, but it must be demoted from "the plan" to "the tested hypothesis" until backtest numbers exist. A working, validated baseline beats an untested novelty every time judges compare notes.

STEP 1 — Full Audit: Resolving the Conflicting Numbers

Method

Every numeric claim below was checked against at least one of: (a) an existing report's own stated methodology and cross-checks, (b) an independent recomputation performed for this review directly against `raw_dataset/sales_train_evaluation.csv`, `raw_dataset/sales_train_validation.csv`, `raw_dataset/calendar.csv`, and `processed_dataset/sales_long_full.parquet` using pandas. All commands and their raw output are preserved in `SUPPORTING_EVIDENCE.md`.

1.1 — 634 vs. 763 (maximum single-day sales)

Resolved: 763 is the authoritative dataset-wide maximum.

- Independently recomputed from `raw_dataset/sales_train_evaluation.csv` (30,490 × 1,941 cells): max = **763**.
- Cross-checked from `raw_dataset/sales_train_validation.csv` (shorter history): max = **763** (same record, so it isn't an artifact of the extra 28 evaluation-only days).
- Cross-checked from `processed_dataset/sales_long_full.parquet`: max = **763** (matches `PROCESSING_REPORT.md`'s independently-validated figure exactly).
- The record belongs to series `FOODS_3_090_CA_3` on day `d_960` = **2013-09-14** (a Saturday, no calendar event, not a SNAP day — an unexplained organic spike, not a data artifact).
- Where 634 comes from:** when max-sales is computed *scoped* to department `FOODS_2` only, the max is exactly 634. It is also, coincidentally, the max within store `TX_1` only. Both are verified directly in `SUPPORTING_EVIDENCE.md`. This is the classic failure mode of a quick groupby check: filtering to one department or store and reporting that number as "the max" without re-checking against the unfiltered table.

- **Action:** use 763 everywhere. If a department- or store-scoped maximum is ever needed again, label it explicitly (e.g., "max within FOODS_2 = 634"), never as an unqualified "the max."

1.2 — ~30M vs. 59,181,090 (processed row count)

Resolved: 59,181,090 is exact, not approximate, and is correct.

- 30,490 series × 1,941 days = 59,181,090 exactly — a multiplication fact, independently re-verified against both the raw wide-format file's row/column counts and the processed Parquet file's actual row count (`len(df) == 59181090`).
- PROCESSING_REPORT.md §11, §13, and §14 all state and cross-verify this figure via independent re-read of the raw CSVs. EDA_REPORT.md §3 states the identical number and confirms it was unchanged going into the EDA stage.
- **Where ~30M might come from:** no project file anywhere contains a "~30M rows" claim (confirmed by full-text search across every .md, .json, and .csv in the project). The most plausible origin is a verbal back-of-envelope estimate that conflated the **series count** (30,490 — often shorthand "30K series") with a **row count**, or assumed roughly 1,000 days of history per series rather than the actual 1,941. It is not a competing calculation from any file in this project — it should simply be dropped in favor of the exact, independently-verified 59,181,090.

1.3 — 69.56% vs. 68.6% (FOODS share of total sales)

Resolved: 68.6% (precisely 68.62%) is correct. 69.56% does not reproduce under any tested definition and should be treated as unverified/incorrect.

Independently recomputed FOODS unit-share three ways for this review:

Source	FOODS units	Total units	FOODS share
raw_dataset/sales_train_evaluation.csv (full history)	45,922,427	66,927,173	68.6155%
raw_dataset/sales_train_validation.csv (28 fewer days)	45,089,939	65,695,409	68.6348%
processed_dataset/sales_long_full.parquet	45,922,427	66,927,173	68.6155%

All three agree to within 0.02 percentage points, which is exactly what should happen given the validation file is a strict prefix of the evaluation file. For completeness, two alternative (and non-standard) definitions were also tested to see if either explained 69.56% — neither does:

- Revenue-weighted (units × price) FOODS share: **58.0%** — much lower, because FOODS items are cheaper on average (mean \$3.25) than HOUSEHOLD/HOBBIES.
- FOODS share of *nonzero rows* (row-count-weighted, not unit-weighted): **56.3%** — also does not match.

Action: use 68.6% (or 68.62% for one more digit of precision) as the authoritative figure everywhere. Drop 69.56% — it is not traceable to any file or reasonable alternate definition in this project and should not be repeated to judges.

1.4 — 42,840 vs. 30,490 (series count)

Resolved: both numbers are correct, but they measure different things — this was never actually a data error, it was a definitional mismatch.

- **30,490** = the number of forecastable store-item series (3,049 items × 10 stores). Independently re-verified: `sales['id'].nunique() == 30490`, `item_id.nunique() == 3049`, `store_id.nunique() == 10`, and every item appears in exactly 10 stores (a perfectly balanced grid, confirmed in DATASET_SUMMARY.md §7). **This is the number that matters for this hackathon** — sample_submission.csv requires exactly one forecast row per store-item series (60,980 rows = 30,490 × 2 ID-suffix blocks), confirmed directly.
- **42,840** = 1 (grand total) + 3 (state) + 10 (store) + 3 (category) + 7 (department) + 9 (state×category) + 21 (state×department) + 30 (store×category) + 70 (store×department) + 3,049 (item) + 9,147 (state×item) + 30,490 (store×item) = **42,840**. This is

public, well-documented background knowledge about the M5 competition's official WRMSSE hierarchy (12 aggregation levels used only for that specific weighted-and-scaled accuracy metric) — it is not a fact independently derived from this project's own files, and it is flagged here as background knowledge, consistent with how `ML_FORECASTING_APPROACH.md` already treats WRMSSE (as unconfirmed for this specific hackathon).

- **Action:** 30,490 is the number to build, train, and predict for. 42,840 only becomes relevant if the team implements full 12-level hierarchical reconciliation or replicates the official WRMSSE metric — neither is required unless the hackathon's actual scoring rubric asks for it (still unconfirmed — see Step 10 / Team Decisions).

1.5 — Other things checked while auditing

- **Zero-sales rate (68.0%, 40,241,819 rows):** independently re-verified from `raw_dataset/sales_train_evaluation.csv` — $40,241,819 / 59,181,090 = 67.9978\% \approx 68.0\%$. Confirmed identical across `PROCESSING_REPORT.md`, `EDA_REPORT.md`, and this review's fresh computation.
- **Missing-price rate (20.78%, 12,299,413 rows):** stated consistently in `PROCESSING_REPORT.md` and `EDA_REPORT.md`; not independently recomputed for this review (would require re-running the full price join), but the two existing reports were built by different stages and agree exactly, which is itself a consistency check worth noting.
- **Series/rows with partial price history (19,558 of 30,490 = 64.1%)** — from `DATASET_SUMMARY.md`, consistent with the "20.78% of rows missing price" figure (a large minority of *series* having *some* missing weeks naturally produces a smaller overall *row-level* missing-price percentage; these are not competing numbers, they answer different questions — series-level vs. row-level).
- `ML_FORECASTING_APPROACH.md`'s **Appendix A / AUDIT_LOG.md** is explicit that it was written in an environment with no access to `raw_dataset/` or `processed_dataset/` — it derived its numbers entirely from the four report PDFs. This review, by contrast, had direct access to both folders and independently recomputed the core statistics rather than only re-reading prior reports. No contradiction was found between what `ML_FORECASTING_APPROACH.md` cited and what this review independently verified.

No numerical claim in this document is presented as fact unless it was traced to a source in the bullet list above or to `SUPPORTING_EVIDENCE.md`.

STEP 2 — Dataset Understanding

Files, at a glance

File	Role	Rows × Cols	Verified
<code>raw_dataset/sales_train_evaluation.csv</code>	Primary sales history, <code>d_1–d_1941</code> (2011-01-29 → 2016-05-22)	30,490 × 1,947	■ re-verified
<code>raw_dataset/sales_train_validation.csv</code>	Strict subset, <code>d_1–d_1913</code> (28 fewer days) — same IDs, values identical on shared days	30,490 × 1,919	■ re-verified subset relationship
<code>raw_dataset/calendar.csv</code>	Maps <code>d_*</code> to real dates; weekday/month/year, events, SNAP flags; extends 28 days *past* the sales data	1,969 × 14	■ re-verified
<code>raw_dataset/sell_prices.csv</code>	Weekly price per store-item; also extends 28 days into the forecast window	6,841,121 × 4	(from existing reports; internally consistent)
<code>raw_dataset/sample_submission.csv</code>	Output template: <code>id, F1...F28</code>	60,980 × 29	(from existing reports)

File	Role	Rows × Cols	Verified
processed_dataset/sales_long_full.parquet	Long-format join of all of the above (the only file that should be used for EDA/feature engineering/training)	59,181,090 × 22	■ re-verified

FACT — What each raw file contains: sales_train_evaluation.csv is one row per store-item series, one column per day, cell = units sold. calendar.csv is one row per calendar date with weekday/month/year/event/SNAP fields, and — critically — it runs 28 days *beyond* the last day of sales data, so the entire forecast window's calendar is already known. sell_prices.csv is one row per (store, item, week), i.e., price is recorded weekly, not daily; it also extends into the forecast window. sample_submission.csv only defines the required *output shape* (bottom-level, store-item granularity) — it carries no sales information.

FACT — What the processed file contains: one row per (series, day) — the wide sales table melted into long format, then left-joined against calendar.csv (on d) and sell_prices.csv (on store_id+item_id+wm_yr_wk). Row count was unchanged through both joins (confirmed no fan-out from duplicate keys), and five independent cross-checks (sum of sales, zero-count, max, spot-checked dates, spot-checked prices) all matched the raw source exactly.

Core numbers (all FACT, independently verified where marked):

- Series: **30,490** ■ (= 3,049 items × 10 stores, a perfectly balanced grid — every item is sold in every store, no exceptions)
- Stores: **10**, across **3 states** (CA: 4 stores, TX: 3, WI: 3)
- Departments: **7**, under **3 categories** (FOODS: FOODS_1/2/3; HOBBIES: HOBBIES_1/2; HOUSEHOLD: HOUSEHOLD_1/2)
- Date range: **2011-01-29 to 2016-05-22** (1,941 days, ~5.3 years), zero gaps ■
- Forecast horizon: **28 days, 2016-05-23 → 2016-06-19** — the true unknown target, contained in no file
- Target variable: sales — non-negative integer units sold, **not revenue** (no dollar sign anywhere in the sales files)
- Calendar info: weekday, month, year, event_name_1/2, event_type_1/2, all non-missing for the entire 28-day forecast window ■
- SNAP info: state-specific daily 0/1 flags (snap_CA/snap_TX/snap_WI), ~33% of days per state, non-missing for the forecast window
- Price info: weekly, non-missing for the forecast window in sell_prices.csv, but **not yet joined into sales_long_full.parquet for the future 28 days** — the processed table only covers d_1–d_1941, so a separate future-price join is needed before this feature can actually be used at forecast time (a real, currently-unaddressed gap — flagged again in Step 7)
- Missing values: **0** in sales, dates, or SNAP columns; 91.86% missing event_name_1/event_type_1 (expected — most days have no event); 20.78% missing sell_price (expected — pre-listing periods, see below)
- Zero-sales rows: **40,241,819 (68.00%)** ■ — the single most defining property of this dataset
- Partial price histories: 19,558 of 30,490 series (64.1%) do not have a price for every week; range 19–282 weeks of price coverage per series
- Product/listing behavior: every item is sold in every store (10/10, no exceptions) — but that does not mean every item was *available* from day one in every store; the price-history gap is the evidence for that (Step 8)

INFERENCE: the missing 20.78% of sell_price values mostly represent pre-listing periods (product not yet stocked), not random data loss — supported by the fact that a spot-checked example series' zero-sales run and no-price run end within days of each other, and by the systematic Phase-3 finding that this holds for 99.48% of series (median gap 3 days). This is not a labeled fact (no "listed" field exists) — it is a well-evidenced interpretation.

ASSUMPTION we recommend the team adopt: treat sales_train_evaluation.csv as the single source of historical truth (it strictly contains everything sales_train_validation.csv has, plus 28 more real days) and stop referencing the validation file for anything except a sanity cross-check.

STEP 3 — Review of the EDA

The EDA (EDA/EDA_REPORT.md, 9 phases, 26 charts, 33 tables, 8 phase-level JSON stat dumps) is methodologically sound: it runs on the full 59.18M-row table (one documented sampling exception, seed=99, n=501 series, used only for a distribution-of-effects view in Phase 8 — clearly labeled), explicitly sorts by (id, date) before any time-dependent computation (necessary because the Parquet file's physical storage order is date-major, not id-major — an easy bug to introduce and this EDA explicitly guards against it), and separates FACT from INTERPRETATION throughout. This review independently re-verified the two headline numbers (zero-rate, FOODS share) and both matched exactly.

3.1 — Strongest findings that actually matter for forecasting

- `days_since_last_sale` → **P(sale today)** is a near-perfect staircase: 65.2% (sold yesterday) → 38.3% (1–3 days dry) → 22.2% (4–7) → 12.7% (8–14) → 6.2% (15–28) → 0.6% (29+ days). This is the single cleanest, strongest relationship in the entire EDA — stronger than any calendar or price signal. **Must become a top-priority feature.**
- `rolling_mean_7` (**r=0.820**) and `rolling_mean_28` (**r=0.807**) are the strongest same-day correlations found — stronger than any single lag (`lag_1` r=0.768 down to `lag_28` r=0.672). **Prioritize rolling windows over raw lags.**
- **Leading zero-block length ≈ leading no-price length** (median gap 3 days, 99.48% of series within 7 days). This is the strongest evidence for a listing-aware approach, and it is genuinely structural, not just correlational noise.
- **SNAP effect (+12.7% overall, +17.3% in FOODS, +32.3% in FOODS_2)** — a high-confidence, low-risk feature precisely because the effect lands exactly where domain knowledge predicts (food spending), which is an internal consistency check, not just a correlation found by chance.
- **Named events have large, opposite-direction effects** (Christmas −99.95%, LaborDay +27.5%) that a single "is_event" binary flag completely destroys (aggregate event effect is only −4.6%, misleadingly small). **Use event_name, never a blunt flag.**
- **Weekend effect (+31.1%)** — strong, clean, consistent across every category. Low risk, high value.
- **Heterogeneity is real, not assumed:** per-series zero% ranges from single digits to >95%; per-series seasonality CV spans a >4x range (0.136 to 0.557); event/price sensitivity is majority- but far from unanimously-directional in the sampled 501 series. This supports segment-aware thinking but does not, on its own, prove a segmented model will out-forecast a well-featured global one.

3.2 — Interesting but not directly useful for the forecast

- The **sales-concentration Lorenz curve** (top 10% of series = 54.4% of units) is important for *evaluation design* (don't let aggregate metrics hide long-tail failure) but is not itself a model input.
- **Yearly growth 2011→2015 confounded with assortment growth** (16,762 → 30,474 active series) is a crucial *warning* about a naive year/trend feature, but the finding itself is descriptive, not a feature.
- **Store-level share range (CA_4 6.2% to CA_3 17.0%)** is fully captured by including `store_id` as a categorical feature — the finding motivates *why* store should be in the model, it isn't itself a new feature.

3.3 — Findings that should NOT be used as stated (unsupported / risky)

- **Price-change → sales lift (+71%/+48.5% mean, but 0% median)** is explicitly flagged by the EDA itself as likely confounded (price changes may coincide with other demand-moving events rather than cause the lift). **Do not present this to judges as "we found promotions."** There is no promotion field in this dataset — full stop.
- **Raw categorical year** will contain forecast-horizon values with no or thin training support and should not be used as a plain category.
- `week_of_year` (52 levels, only ~5 full years of data) risks overfitting — real effect, high cardinality relative to data volume.
- **Sampled per-series event/price sensitivity (n=501)** describes the *distribution* of behavior across series, not any specific series — do not use it to justify a per-series causal claim.

STEP 4 & 5 — Critical Review of Proposed Ideas

Ratings: **A = MUST KEEP, B = GOOD/TEST IT, C = OPTIONAL IF TIME, D = DROP.**

#	Idea	Rating	Why
1	Leading Zero / Pre-launch detection	A	Directly evidenced (99.48% alignment, median 3-day gap). Cheap to compute (first_date with non-null sell_price), low leakage risk if computed from pre-origin history. This is the strongest, most defensible piece of the team's novelty.
2	Ghost Stockout Detection	C	No inventory field exists — the dataset genuinely cannot confirm a stockout (ML_FORECASTING_APPROACH.md already states this correctly). Framing it as a *feature that flags "unusual" zero patterns* (not a "we detected real stockouts" claim) is defensible as an experiment; framing it as a confirmed capability is not. Test it, but keep the language honest to judges — see Step 15.
3	Potential Promotion / Price-Shock Detection	B, framed cautiously	Real signal exists (price changes correlate with sales moves) but is explicitly confounded per the EDA's own before/after analysis (median effect = 0%). Usable as a weak "something changed" signal or wider-uncertainty flag — not as a labeled promotion feature. Never claim "we detected promotions."
4	SNAP + Weekend interaction	B	Both main effects are independently strong and clean (SNAP +12.7%/+17.3% FOODS; weekend +31.1%). An interaction term is cheap to test and plausible (grocery trips cluster on SNAP-active weekends) but has not itself been tested in the EDA — must be validated, not assumed to help.
5	Event features (event_name)	A	Strong, specific, calendar-known-in-advance signal. Cheap. Must use identity, not a binary flag.
6	Christmas closure logic	B	Christmas is -99.95% vs. local baseline — close enough to "stores closed" that a hard override (predict ≈0 for Christmas) is a reasonable, low-risk special case. Worth testing as a targeted override rather than trusting the general model to learn one extreme day from ~5 examples in training.

#	Idea	Rating	Why
7	Global LightGBM	A	The correct primary baseline model. Scales to 30,490 series with pooled statistical strength, handles categoricals and non-linearities natively, fast to iterate on.
8	LightGBM + Tweedie objective	A (test against baseline)	Tweedie loss is designed for exactly this shape of target (non-negative, zero-inflated, right-skewed). Directly motivated by Finding 1 (68% zeros). Must still be validated against a standard-objective LightGBM — "sounds right on paper" is not the same as "measured to help on this data."
9	Hurdle / two-stage forecasting	A (test against single model)	The strongest structural response to zero-inflation, and it maps directly onto the two cleanest EDA findings (68% zero rate; the days-since-last-sale staircase belongs naturally in Stage 1). This is the correct centerpiece of the team's novelty — but it is unproven until backtested against a single combined model (Experiment 3 vs. 2 in Step 12).
10	Recency-state features (days_since_last_sale, zero_streak_length)	A	The single strongest relationship in the whole EDA. Must be in the first feature build, computed strictly pre-origin.
11	Hierarchical reconciliation	D for the hackathon; C if time remains after everything else works	sample_submission.csv only requires bottom-level (store-item) forecasts — there is no requirement to reconcile to store/category/state totals. Full reconciliation is a real technique but adds real implementation complexity for a benefit that isn't required by the deliverable. Do not build this before a working bottom-level model exists.
12	AI Supply Chain Copilot	C, and only after Step 14's "must-have" list is done	Interesting demo value for judges, zero forecasting-accuracy value. Must not consume time that the core model needs. See Step 14 — this is explicitly ranked below a working, validated model.

#	Idea	Rating	Why
13	Cannibalization / graph-based modeling	D	No EDA evidence was gathered for cross-item substitution effects at all — this would be building a novelty on zero measured evidence, which is exactly what Step 4 of this review was asked to flag. High implementation difficulty, high leakage risk (co-purchase/substitution features are easy to build with future information by accident), and it directly contradicts the "don't keep something merely because it sounds innovative" instruction.
14	Bullwhip effect	D	Not evidenced anywhere in this project's EDA, and the dataset (raw retail sell-through, not a multi-echelon supply chain with visible upstream orders) does not obviously contain the information needed to measure it. Sounds impressive, has no evidentiary basis here.
15	Foods-first optimization	B, as an evaluation lens, not a modeling shortcut	FOODS is 68.6% of volume and skews the loss of any pooled global model — worth explicitly tracking FOODS vs. non-FOODS metrics separately (Step 12), and worth considering category-aware sample weighting if the global model underperforms on HOBBIES/HOUSEHOLD. But do not literally build "a FOODS-only model and ignore the rest" — the deliverable requires all 30,490 series.
16	Deployment / API / dashboard	C, explicitly last	Zero value if the forecast underneath it is not validated. See Step 14.

STEP 6 — The ML Strategy

Models compared

Model	What it is	Verdict
Model 0 — Naive baseline	Seasonal-naive: predict day *h* of the horizon using the same weekday's sales from a recent prior period (e.g., the mean of the last 4–6 occurrences of that weekday).	Build this. Always. It costs almost nothing to implement, and every later model must beat it or the added complexity has no justification.
Model 1 — Global LightGBM	One LightGBM regressor, standard MSE/MAE-style objective, trained on all 30,490 series pooled together with standard lag/rolling/calendar/price/hierarchy features.	Build this second — it is the real baseline for judging the novelty.

Model	What it is	Verdict
Model 2 — Global LightGBM + Tweedie objective	Same features as Model 1, objective='tweedie'.	Build this third. Directly tests whether a zero-inflation-aware loss function alone (without restructuring into two stages) already captures most of the benefit.
Model 3 — Two-stage hurdle	Stage 1: LightGBM classifier, $P(\text{sales} > 0)$. Stage 2: LightGBM regressor (Tweedie or log-target), $E(\text{units} \mid \text{sales} > 0)$, trained only on nonzero rows. Final = $P(\text{sale}) \times E(\text{units} \mid \text{sale})$.	Build this fourth, and only if Models 1–2 are already working and backtested. This is the proposed novelty's structural core.

Recommendation: implement Models 0–2 first, in that order, each fully backtested before moving to the next. Model 3 (the hurdle) is the team's actual novelty and should absolutely be attempted, but its value must be demonstrated as an improvement *over* Model 2, not assumed. If time runs out after Model 2, the team still has a complete, competitive, honestly-validated submission — this is the fallback the hackathon plan must protect (see Step 14).

Do not start with the hurdle model. Building the most complex model first means that if it underperforms, there is no time left to fall back to something simpler and provably correct — and no ablation evidence to explain *why* it underperformed.

STEP 7 — Final Feature Engineering Plan

Every feature below states what it represents, why it should help, whether it's safe for a 28-day-ahead forecast, and its leakage risk. **"Safe" always means: computed using only information available at the fixed forecast origin (the last known day, d_{1941}), then held constant across all 28 target days** — see Step 10 for why this constraint exists.

A. Historical demand features

Feature	Represents	Why it helps	28-day-safe?	Leakage risk
lag_1, lag_7, lag_14	Sales 1/7/14 days before the *origin*	Strong same-day correlation ($r=0.77/0.72/0.69$) *at the origin*	Only if computed once relative to the origin and held constant across all 28 horizon days	HIGH if recomputed per-target-day — that would require future sales
lag_28	Sales exactly 28 days before the origin	Still meaningfully correlated ($r=0.672$); structurally the only lag that never reaches into the forecast window for any of the 28 target days	Yes, safe by construction	Low

B. Recency features

Feature	Represents	Why it helps	Safe?	Leakage risk
rolling_mean_7, rolling_mean_28	Mean sales over the 7/28 days ending at the origin	Strongest same-day predictors found ($r=0.82 / 0.81$)	Yes, if the window ends at the origin and is held constant across the horizon	HIGH if window is allowed to slide into the forecast period
rolling_std_7 / rolling_std_28	Recent volatility	Series CV is highly dispersed ($>50\%$ of series have $CV>2$); not yet directly correlation-tested against the target, so treat as medium-confidence	Same origin-window rule	Same as above

C. Zero/intermittent-demand features

Feature	Represents	Why it helps	Safe?	Leakage risk
days_since_last_sale	Days since the last nonzero sale, as of the origin	Single cleanest relationship in the whole EDA (65.2%→0.6% staircase)	Yes, if computed strictly from pre-origin history (a running count)	Low, if origin-bounded
zero_streak_length	Current consecutive-zero run length, as of the origin	Same underlying signal in run-length form; directly usable for hurdle Stage 1	Yes, same rule	Low, if origin-bounded
recent_nonzero_rate (e.g., % nonzero in trailing 28 days)	Recent activity frequency	Useful for very low-volume series where the mean is near zero anyway	Yes, trailing window ending at origin	Low, if origin-bounded
activity_class (High/Regular/Sparse/Extremely-sparse)	Data-driven segment label from the per-series zero% distribution	Confirmed real heterogeneity across segments	Must be computed from pre-origin history only	HIGH if computed on full series history — would leak the forecast period's own behavior into its own label

D. Product/listing features

Feature	Represents	Why it helps	Safe?	Leakage risk
days_since_first_listing / pre-listing flag	Days since the series' first non-null sell_price	Strongest evidence in the EDA for a structurally distinct zero type (99.48% alignment with leading-zero blocks)	Yes — "first priced date" is a historical fact fixed well before the origin for the overwhelming majority of series	Low. Edge case: a series whose first-ever listing falls *inside* the 28-day forecast window needs care (see caveat below)

Caveat, stated plainly: this feature identifies *past* pre-listing periods. It does **not** tell us whether a currently-unlisted or brand-new item will be listed at some point during the 28-day forecast window — the dataset gives no forward listing calendar. Treat any forecast for a series with a very short or currently-zero pre-origin history as inherently higher-uncertainty, not as something this feature "solves."

E. Price features

Feature	Represents	Why it helps	Safe?	Leakage risk
current_price	Price on the target day	Legitimately forward-known in this dataset (sell_prices.csv covers the future window)	Yes, but sales_long_full.parquet as built does not yet include future-horizon price rows — a separate join against sell_prices.csv is required before this feature is usable at forecast time	Low once joined correctly; currently a build gap, not a leakage risk
price_relative_to_recent_average	Price ÷ the item's own trailing average price	Clearer relationship with demand than raw price once cross-item scale is removed (2.34 units at <0.85x avg price → 1.29 units at 1.0–1.05x, non-monotonic above that)	Yes, if "own average" is a trailing/expanding window ending at the origin	HIGH if the average uses full-series history including the forecast period

Feature	Represents	Why it helps	Safe?	Leakage risk
price_change_pct / potential_promotion_signal	Recent price-change magnitude/direction	Weak, explicitly-flagged-a s-confounded demand-shock signal (Step 9)	Yes, mechanically	Low mechanically, but causally unreliable — do not treat as a validated driver

F. Calendar features

Feature	Why it helps	Safe?
day_of_week / is_weekend	+31.1% weekend effect, consistent across every category	Yes — fully known for the entire horizon
month	Real but modest (8.99% high-low spread)	Yes
Continuous time index (not raw categorical year)	Avoids the unseen-future-category problem while still allowing a trend signal	Yes, but must be interpreted alongside the composition-effect caveat (Step 3) — a time trend on this data partly reflects catalog growth, not pure organic demand growth

G. Event features

Feature	Why it helps	Safe?
event_name (specific identity, not a binary flag)	Christmas –99.95% vs. LaborDay +27.5% — a blunt flag destroys this signal entirely	Yes — full calendar known in advance
event_type_1	Coarser grouping, modest effect (Sporting 1.174 vs. National 0.966)	Yes

H. SNAP features

Feature	Why it helps	Safe?
State-matched snap_indicator (row's own state)	+12.7% overall, +17.3% FOODS, +32.3% FOODS_2 — a genuine, internally-consistent signal	Yes — full SNAP calendar known in advance

I. Store/category hierarchy features

Feature	Why it helps	Safe?
item_id, dept_id, cat_id, store_id, state_id	Substantial, consistent behavioral differences at every level (zero% ranges 58.6%–88.4% by department alone)	Yes — static, always known

J. Novelty features

Feature	Why it helps	Safe?
Pre-listing flag / days-since-first-listing (see D)	Core of the listing-aware novelty	Yes, with the caveat above
Hurdle-model structure itself (not a "feature" per se, but a modeling choice)	Structural response to 68% zero-inflation	N/A — a model architecture decision, tested in Step 12

STEP 8 — Zero-Sales Strategy

We cannot know which zero is a stockout. There is no inventory field anywhere in the five raw files — this was independently confirmed by a full column scan documented in DATASET_SUMMARY.md and raw_dataset/Dataset_Explanation/DATASET_EXPLAINED.md, and this review found nothing to contradict it.

Four types of zero, and what to do with each:

- **Pre-launch / not-yet-listed zero.** FACT: leading zero-runs align with leading no-price periods for 99.48% of series (median gap 3 days). INFERENCE: this most likely reflects "not yet stocked," not "stocked but no demand." **Recommendation: do not remove these rows.** Instead, flag them with a `pre_listing` indicator (derived from the first non-null `sell_price` date) and let the model condition on it — the hurdle Stage 1 classifier can learn that this state has a near-zero sale probability, which is both true and useful, without deleting real, correctly-recorded observations.
- **Genuine intermittent-demand zero.** This is the **normal, expected** case once a series is active — a listed, available product simply not purchased that day. **Recommendation: keep as-is, this is the primary target signal for the whole problem.** `days_since_last_sale` and `zero_streak_length` are the features designed to help the model reason about this state.
- **Potential stockout-like zero.** ASSUMPTION-laden territory: some zeros mid-series, following a long run of nonzero sales, might reflect an out-of-stock event rather than demand collapse — but the dataset provides no way to confirm this. **Recommendation: do NOT invent a stockout label.** If time allows, test a "ghost stockout" **feature** (e.g., an unusually long zero-streak immediately following sustained nonzero sales, relative to that series' own typical streak length) as an experimental input — but always describe it to judges as "a flag for statistically unusual zero patterns," never as "we detected real stockouts."
- **Holiday/event-related zero.** Christmas is -99.95% vs. local baseline — functionally "stores closed." **Recommendation: this is best handled as a calendar feature (`event_name`) or a targeted override (Step 4, item 6), not as part of the zero-type taxonomy above** — it's driven by the calendar, not by listing or demand state.

Bottom line recommendation: retain all zero rows exactly as recorded (already the team's stated position in PROCESSING_REPORT.md, and correct). Build zero-**aware features** (pre-listing flag, recency state, optionally a stockout-suspicion flag) rather than deleting or reinterpreting any observation. This is both the more defensible approach and the one already reflected in the existing processed dataset — no rework needed there.

STEP 9 — Promotion / Price-Shock Strategy

FACT: there is no promotion, discount, markdown, or "on-deal" field anywhere in the five raw files (confirmed by a full column-by-column scan, documented independently in both DATASET_SUMMARY.md and raw_dataset/Dataset_Explanation/DATASET_EXPLAINED.md).

Can price changes be used as a proxy? **Cautiously, and only as a weak signal — not as a confirmed promotion label.** The EDA's own before/after analysis is the reason for the caution: mean sales rose after **both** price increases (+71.0%) and price decreases (+48.5%), while the **median** change in both directions was 0%. That pattern is inconsistent with a simple "price drop → more sales" causal story, and is far more consistent with price changes often coinciding with **something else** that's moving demand (a new item ramping up, a seasonal shift) rather than causing the shift themselves.

Recommended concepts, with limitations stated up front:

- `price_change_pct` — the size and direction of the most recent price move. Mechanically leakage-safe (a historical fact, if computed pre-origin), but **causally unreliable** — treat as a weak, exploratory input, not a validated driver.
- `price_shock` (a binary flag for an unusually large relative price move) — same caveat.
- `potential_promotion_signal` — deliberately named to avoid overclaiming. If used, this should be framed to judges exactly as it's framed in this document: "we cannot confirm promotions exist in this data; this is a price-change-derived proxy, tested for whether it improves forecast accuracy, not a claim that we detected real promotional events."

What NOT to do: do not call a price drop "a promotion" in any deliverable, slide, or code comment. Do not build a feature that assumes causality where the team's own EDA explicitly found the opposite (median-zero effect, most series unaffected). If the ablation study (Step 12) shows this feature doesn't measurably improve backtest accuracy, drop it — its marginal cost (one more feature, one more assumption to defend to judges) is not worth carrying if it isn't earning its place.

STEP 10 — Validation Strategy (Critical)

Random train/test splitting is wrong for this problem and must not be used. A random split lets the model see, e.g., day 1500's sales for a series while predicting day 1200 for the same series — using future information to explain the past. It would also let a model "learn" a series' typical value from scattered future rows even for a fixed date, badly overstating how well it will generalize to a genuinely unseen future window.

Fixed-origin, rolling-origin backtesting

The correct simulation of the real task is: **pick a historical cutoff date T , train using only data up to and including T , generate a 28-day forecast for $T+1 \dots T+28$, and compare against the actual sales we already have on record for that window** (this dataset conveniently gives at least one such window for free: cut at d_{1913} , validate against d_{1914} – d_{1941} , which recreates the exact shape of the real 28-day task using known answers).

	Contains
TRAINING DATA	All observations with date $\leq T$
VALIDATION DATA	The 28 real, already-observed days immediately after T ($T+1 \dots T+28$) — used to score candidate models, never trained on
FINAL UNKNOWN FORECAST DATA	2016-05-23 $\rightarrow$ 2016-06-19 (d_{1942} – d_{1969}) — genuinely unknown, contained in no file, produced only once, at the very end

Why multiple windows, not just one: a single 28-day window might happen to avoid (or include) an unusual event like Christmas, making the result lucky or unlucky rather than representative. **Recommendation: use rolling-origin validation with at least 2–3 historical cutoffs** (e.g., one ending in an ordinary month, one spanning a major holiday) rather than a single fixed split, time permitting. If time is short, one well-chosen window (d_{1913} cutoff, since it's the one window with genuinely revealed ground truth already built into the data) is the non-negotiable minimum.

How features must be calculated without seeing the future: every lag/rolling/recency/price-relative/activity-class feature (Step 7) must be computed using only date $\leq T$, then held constant across all 28 target days of that backtest window — never recomputed per-target-day using dates inside the window. This is the single biggest correctness risk in the entire project, flagged independently and consistently by both EDA_REPORT.md §14 and ML_FORECASTING_APPROACH.md, and this review agrees it deserves that level of emphasis. A model that "accidentally" uses $T+5$'s actual sales to help predict $T+3$ will look deceptively good in backtesting and then fail in the real, truly-unknown forecast — exactly the failure mode a hackathon judge is most likely to probe for.

Direct/fixed-origin vs. recursive forecasting — pick one deliberately:

- **Direct (recommended for the hackathon):** compute every feature once relative to the origin, predict all 28 days at once. Simpler, faster, and avoids compounding one-step-ahead prediction errors across 28 recursive steps. lag_{28} is the only single lag that is naturally safe for every horizon day under this approach without special handling.
- **Recursive:** predict day 1, feed that prediction back in as if it were lag_1 for day 2, and so on. More complex, error-compounding, and harder to backtest correctly under time pressure. **Not recommended as the primary approach for this hackathon** given the time constraints in Step 14 — direct multi-horizon is both simpler to implement correctly and easier to explain to judges.

Fair comparison across models: use the *same* backtest window(s), the *same* evaluation metric(s), and the *same* feature-availability rules for every candidate (Model 0 through Model 3) — never let a "GOOD" result on one model come from it having access to information another model didn't get.

STEP 11 — Novelty

Candidate novelty directions (evaluated independently, not just inherited)

1. Listing-Aware + Recency-Aware Demand Forecasting (the team's proposed direction)

- **Problem it solves:** treats "not yet listed" zeros and "listed but currently in a dry spell" zeros as the structurally different things the EDA shows them to be, instead of feeding a model 68% identical-looking zeros.
- **Evidence from the dataset:** two independently strong findings converge on this — (a) leading-zero/leading-no-price alignment (99.48% of series within 7 days), and (b) the days-since-last-sale staircase (the cleanest relationship in the whole EDA). Both are FACT, not hypothesis.
- **Technical implementation:** a `pre_listing` flag (from first-priced-date) feeding Stage 1 of a hurdle model, plus `days_since_last_sale/zero_streak_length` as first-class features in both stages.
- **Why a naive model might miss it:** a single model trained on all zeros identically has no way to learn that a pre-listing zero and a day-47-of-a-dry-spell zero mean very different things for tomorrow's probability of a sale — it can only average them together.
- **How to prove it helps:** ablation Experiments 3→5 in Step 12 — compare backtest accuracy with and without each component, isolating the marginal contribution of each.
- **Explained to judges in plain English:** *"About 20% of this dataset's 'no sales' rows are really 'not on the shelf yet,' not 'nobody wanted it.' We taught the model to tell the difference, and used how long it's been since a product last sold as its own feature — because that turned out to be the single best predictor of whether it sells tomorrow."*

Independent verdict: defensible, genuinely evidenced, and worth being the headline story — but it is currently a well-supported hypothesis, not a proven result. Nothing in this review found reason to reject it; it should simply be held to the same "prove it in backtest" standard as everything else.

2. Two-stage hurdle structure itself, as a standalone novelty framing

- Sometimes worth separating from novelty #1: even without listing-awareness, splitting $P(\text{sale})$ from $E(\text{units} \mid \text{sale})$ is a structural response to 68% zero-inflation, on its own defensible territory (standard in intermittent-demand forecasting literature, e.g. Croston-family methods) — but it is a well-known technique, not something unique to this team's analysis.
Recommendation: fold this into novelty #1 as "part of the mechanism," not a separate headline — the genuinely dataset-specific insight is the listing-awareness and the recency-staircase, not the hurdle structure by itself.

3. Event-identity-aware forecasting (not a generic holiday flag)

- Real, evidenced (Christmas −99.95% vs. LaborDay +27.5%, masked by a misleading −4.6% aggregate). **Verdict: a legitimate B-tier supporting feature, not strong enough alone to be the primary novelty** — it's a well-established technique (using specific event dummies instead of one flag) rather than something distinctive to this project's own findings.

Recommended primary novelty

"Listing-Aware + Recency-Aware Demand Forecasting" (candidate #1) is the correct choice, on the strength of two independently-verified, dataset-specific findings rather than one. It should be presented to judges exactly as this document frames it: a tested hypothesis with strong supporting evidence, validated (or refined) via the ablation plan below — not a foregone conclusion.

STEP 12 — Ablation / Experiment Plan

#	Experiment	Adds	What improvement this is meant to demonstrate
0	Naive seasonal baseline	—	The floor every other model must beat
1	Global LightGBM, calendar features only (weekday/month/event/SNAP)	Calendar	Whether calendar signal alone materially beats naive

#	Experiment	Adds	What improvement this is meant to demonstrate
2	+ Price features	current price, price_relative_to_recent_average	Whether price adds value once calendar is already in the model
3	+ Recency features	days_since_last_sale, zero_streak_length, rolling_mean_7/28	Tests whether the single strongest EDA relationship translates into forecast accuracy gain
4	+ Zero-state / listing-aware features	pre_listing flag, activity_class	Tests the core novelty claim directly
5	Switch objective to Tweedie (on top of Exp. 4's feature set)	Tweedie loss	Isolates whether the *loss function* (not just features) meaningfully helps a zero-inflated target
6	Two-stage hurdle (same final feature set as Exp. 5, restructured into Stage1×Stage2)	Structural change	Tests whether explicitly separating occurrence from magnitude beats a single well-featured model
7	+ SNAP/Event interaction, price-shock signal	Interaction + weak signals	Tests the marginal, likely-small contribution of the more speculative features from Step 4/5

Why this ordering matters: each experiment adds exactly one thing on top of the last, so a backtest accuracy change between consecutive experiments can be attributed to that one addition — this is what makes it possible to honestly say “we tested each hypothesis, we didn’t just add every feature we could think of.”* If time is short, Experiments 0–4 are the non-negotiable minimum (they cover the baseline and the core novelty claim); 5–7 are valuable but skippable refinements — see Step 14 for the priority ordering under time pressure.

STEP 13 — Final Recommended Project Architecture

Raw Dataset	[DONE – verified untouched]
Data Validation	[DONE – PROCESSING_REPORT.md]
Processed Dataset (sales_long_full.parquet)	[DONE – 59,181,090 x 22, re-verified]
Feature Engineering	[MUST HAVE – Step 7, origin-relative only]
Training Dataset (with train/valid split per Step 10)	[MUST HAVE]
Baseline (Model 0, seasonal-naive)	[MUST HAVE]
Candidate Models (Model 1 -> 2 -> 3)	[MUST HAVE through Model 2; Model 3 MUST HAVE if time allows]
Validation (fixed-origin backtest, >=1 window; 2-3 if time allows)	[MUST HAVE]
Best Model Selection (by backtest metric, not intuition)	[MUST HAVE]
28-Day Forecast (2016-05-23 to 2016-06-19, formatted to sample_submission.csv shape)	[MUST HAVE]
Risk/Anomaly Analysis (flag low-confidence series: sparse/new/highly volatile)	[NICE TO HAVE]
AI Copilot / Dashboard	[ONLY IF TIME – after everything above is done and validated]

STEP 14 — Hackathon Feasibility

Component	Difficulty
Feature engineering (Groups A–I, origin-relative)	MEDIUM
Model 0 (naive baseline)	LOW
Model 1 (global LightGBM)	LOW–MEDIUM
Model 2 (+ Tweedie)	LOW (same pipeline, one parameter)
Fixed-origin backtest harness	MEDIUM (easy to get subtly wrong — see Step 10)
Model 3 (hurdle, two-stage)	MEDIUM–HIGH (two models to train/tune, plus combining logic)
Pre-listing / listing-aware feature	LOW (already have the "first priced date" concept from EDA Phase 3)
Ablation study (Step 12)	MEDIUM (mostly a time cost — re-running the same harness 6–8 times)
28-day final forecast + submission formatting	LOW, once the pipeline exists
Dashboard / API / GenAI copilot	MEDIUM–HIGH, and irrelevant if the model underneath isn't validated

Suggested priority order (day-by-day, adapt to actual hackathon length)

- **Day 1:** Feature engineering (Groups A, B, C, F, G, H, I — the "must have" list from Step 7). Build the fixed-origin backtest harness and validate it on the `d_1913` cutoff window using Model 0 (naive) as the very first thing that runs end-to-end. **Goal: a working, if crude, submission pipeline by end of Day 1.**
- **Day 2:** Model 1 (global LightGBM), then Model 2 (Tweedie). Run Experiments 1–4 from Step 12. This is the point where the team has a real, honestly-backtested, competitive submission — protect this checkpoint.
- **Day 3:** Model 3 (hurdle) and the listing-aware feature (Group D), Experiments 5–6. If this beats Model 2 in backtest, it becomes the final model; if it doesn't, **ship Model 2 and say so honestly** — a smaller, proven improvement beats an unproven bigger one.
- **Day 4 (or remaining time):** Experiment 7 (interactions, price-shock), Christmas override, risk/anomaly flagging, then — only if genuinely spare time remains — a lightweight dashboard or copilot layer on top of the already-working forecast.

The non-negotiable rule: at every checkpoint, the team must have a complete, working, submission-shaped forecast from the *best model validated so far*. Never let "we're about to try something better" replace "we have something that works right now."

STEP 15 — Final Team Recommendation

1. What should we definitely build? A global LightGBM baseline (Model 1), backtested correctly with origin-relative features on at least the `d_1913` cutoff window, and the recency-state features (`days_since_last_sale`, `zero_streak_length`, `rolling_mean_7/28`). These are the highest-confidence, highest-evidence, lowest-risk components in this entire review.

2. What should we test, not assume? The Tweedie objective, the hurdle structure, and the listing-aware pre-listing flag. All three are well-motivated by the EDA — none of the three has been measured yet. Test each via the Step 12 ablation before claiming any of them "worked."

3. What should we avoid? Cannibalization/graph-based modeling and "bullwhip effect" analysis (zero evidentiary basis in this project's own EDA); presenting price changes as confirmed promotions; presenting the pre-listing/stockout distinction as something the data can definitively prove; hierarchical reconciliation before a working bottom-level model exists; building the dashboard/copilot before the forecast underneath it is validated.

4. Our strongest novelty: Listing-Aware + Recency-Aware Demand Forecasting — genuinely evidenced by two independent, strong EDA findings (leading-zero/no-price alignment; the days-since-last-sale staircase), not invented to satisfy a hackathon requirement.

5. Our baseline model: seasonal-naive (Model 0) first, then a single global LightGBM with standard MSE-family loss (Model 1) as the real bar to clear.

6. Our final candidate model: LightGBM with Tweedie objective (Model 2) at minimum; the two-stage hurdle model with listing-aware and recency features (Model 3) if — and only if — it demonstrably beats Model 2 in backtest.

7. Features we should definitely create: `days_since_last_sale`, `zero_streak_length`, `rolling_mean_7`, `rolling_mean_28`, `lag_28` (origin-safe by construction), `pre_listing` flag, `event_name`, state-matched `snap_indicator`, `is_weekend`, full hierarchy fields (`item_id/dept_id/cat_id/store_id/state_id`).

8. Assumptions that are risky and must be labeled as such to judges: that pre-listing zeros are definitely "not yet listed" (well-evidenced, not proven — no labeled field exists); that any "stockout" feature reflects real stockouts (it does not — there is no inventory field); that price changes reflect promotions (the team's own EDA found this is likely confounded); that a two-stage hurdle model will beat a single well-featured model (plausible, untested at time of writing).

9. Claims we should NOT make to judges:

- "We detected real promotions in the data" — false, no promotion field exists.
- "We identified actual stockouts" — false, no inventory field exists.
- "Sales are declining/growing organically year over year" — misleading without the assortment-growth caveat.
- "634 is the maximum sale" or "69.56% of sales are FOODS" or "there are 42,840 series to forecast" or "the dataset has ~30M rows" — all four are the resolved/incorrect figures from Step 1; use the corrected values (763, 68.6%, 30,490, 59,181,090) instead.
- "Our novelty is proven to improve accuracy" — not yet true at the time of writing; say "our novelty is evidenced by the EDA and we backtested it, showing X% improvement" only once that backtest actually exists.

10. What would make this project genuinely competitive: an honestly-reported backtest table (Model 0 through Model 3, same window, same metric) showing exactly how much each component of the novelty actually contributed — that table, more than any single clever feature, is what turns "we had a lot of good ideas" into "we proved which ideas were right." Judges see a lot of hackathon teams claim sophistication; a team that can show a clean ablation table and say plainly "component X didn't help, so we dropped it" reads as more credible, not less.

Final Validation of This Document

Check	Result
Every disputed number (634/763, ~30M/59,181,090, 69.56%/68.6%, 42,840/30,490) resolved with a stated cause, not silently picked	■ Step 1
Core statistics independently re-verified directly against <code>raw_dataset/</code> and <code>processed_dataset/</code> , not only re-read from prior reports	■ (see <code>SUPPORTING_EVIDENCE.md</code> for exact commands/output)
No file in <code>raw_dataset/</code> , <code>processed_dataset/</code> , <code>EDA/</code> , <code>Project_Approach/</code> , or <code>analysis_output/</code> modified while preparing this document	■ read-only throughout
No feature-engineering recommendation uses information unavailable at the forecast origin	■ Step 7, explicit per-feature leakage column
No unsupported claim presented as fact (FACT/INFERENCE/ASSUMPTION labeled throughout)	■
Hackathon feasibility grounded in a realistic priority order with a protected fallback at every stage	■ Step 14

End of FINAL_PROJECT_APPROACH.md. See `SUPPORTING_EVIDENCE.md` for the full numeric audit trail.